
Documentation du Projet de Jeu de Dés

Table des Matières

1. Introduction
2. Prérequis
3. Installation
4. Configuration
5. Utilisation
6. Structure du Projet
7. Endpoints API
8. Composants
9. Fonctionnalités
10. Fonctionnalités en Cours de Développement
11. Tests
12. Contribution
13. Licence

1. Introduction

Ce projet est un jeu de dés développé en React pour le frontend et Express.js pour le backend, avec une gestion de scores et sessions stockées dans une base de données MySQL. Un backend additionnel en Python utilisant Django a été ajouté. Le jeu permet aux utilisateurs de lancer des dés, enregistrer les scores, et terminer des sessions de jeu. Les scores sont affichés dans une page dédiée.

2. Prérequis

- Node.js et npm
 - MySQL
 - Python et pip
 - Un navigateur web moderne
-

3. Installation

Installation via Docker

- **Clonez le dépôt :**

bash

Copier le code

```
git clone https://github.com/KevinOlinga/dice-game.git
cd dice-game
```

- **Assurez-vous que Docker est installé :**

- **Démarrez les services via Docker Compose :**

``bash``

Copier le code

```
docker-compose up --build
```

Cette commande construit les images Docker pour le frontend, le backend Express, le backend Django, et la base de données MySQL, puis démarre les conteneurs associés.

Remarque : Docker Compose gère le démarrage des services dans le bon ordre, le déploiement des images et la gestion des volumes pour persister les données.

Installation Manuelle

Installez les dépendances pour le backend Express :

Copier le code

```
cd dice-game-backend
npm install
```

Installez les dépendances pour le frontend :

Copier le code

```
cd ../dice-game-frontend
npm install
```

Installez les dépendances pour le backend Django :

bash

Copier le code

```
cd ../deConfig/deConfig
```

```
pip install -r requirements.txt
```

- **Configurez la base de données MySQL :**

- Assurez-vous que MySQL est installé et en cours d'exécution. (MySQL 8.4 de préférence)

Importez la base de données :

bash

Copier le code

```
mysql -u root -p < database/database_backup.sql
```

-

4. Configuration

Docker

- **Les fichiers Docker ont été ajoutés pour chaque partie du projet :**

- `Dockerfile` pour le backend Express
(`/dice-game-backend/Dockerfile`)
- `Dockerfile` pour le frontend React
(`/dice-game-frontend/Dockerfile`)
- `Dockerfile` pour le backend Django
(`/deConfig/deConfig/Dockerfile`)
- `Dockerfile` pour le serveur MYSQL
(`/deConfig/deConfig/Dockerfile`)
- `docker-compose.yml` pour orchestrer les conteneurs
(`/docker-compose.yml`)

Variables d'environnement

1. Backend Express :

Créez un fichier `.env` à la racine du dossier `dice_game_backend` et ajoutez-y les variables suivantes :

env

Copier le code

`DB_HOST=votre serveur`

`DB_USER=root`

`DB_PASSWORD=votre-mot-de-passe`

`DB_NAME=dice_game`

`PORT=3001`

○

2. Backend Django :

Créez un fichier `.env` à la racine du dossier `deConfig/deConfig` et ajoutez-y les variables suivantes :

env

Copier le code

`DB_HOST=votre serveur`

`DB_USER=root`

`DB_PASSWORD=votre-mot-de-passe`

`DB_NAME=dice_game`

`PORT=8000`

○

5. Utilisation

Avec Docker

Démarrez les services via Docker Compose :

bash

Copier le code

`docker-compose up --build`

Accédez à l'application dans votre navigateur :

○ <http://localhost:3000>

Installation Manuelle

Démarrez le serveur backend Express :

bash

Copier le code

```
cd dice_game_backend  
node app.js
```

Démarrez le serveur backend Django :

bash

Copier le code

```
cd ../deConfig/deConfig  
python manage.py runserver
```

Démarrez l'application frontend :

bash

Copier le code

```
cd ../dice_game_frontend  
npm start
```

Accédez à l'application dans votre navigateur :

- <http://localhost:3000>
-

6. Structure du Projet

```
/dice-game  
|  
├── /dice-game-backend  
|   ├── /node_modules  
|   ├── .env  
|   ├── Dockerfile  
|   ├── app.js  
|   ├── package.json  
|   └── ...  
|  
├── /dice-game-frontend  
|   ├── /src  
|   |   ├── /components  
|   |   |   ├── DiceGame.js  
|   |   |   ├── Scorepage.js  
|   |   |   ├── Header.js  
|   |   |   ├── Footer.js  
|   |   |   ├── Login.js  
|   |   |   └── SessionForm.js
```

```
| | | └─ /assets
| | |   └─ images
| | |     └─ dice-face-1.png
| | |     └─ dice-face-2.png
| | |     └─ dice-face-3.png
| | |     └─ dice-face-4.png
| | |     └─ dice-face-5.png
| | |     └─ dice-face-6.png
| | └─ App.js
| | └─ index.js
| └─ /public
|   └─ index.html
|   └─ ...
|
└─ /deConfig
  └─ /deConfig
    └─ /app
      └─ /migrations
      └─ /models
      └─ /views
      └─ ...
    └─ .env
    └─ Dockerfile
    └─ manage.py
    └─ requirements.txt
    └─ ...
|
└─ /database
  └─ dice_game.sql
  └─ init-db.sql
└─ /docker-compose.yml
└─ README.md
```

7. Endpoints API

Express Backend

- **Créer une session** : `POST /sessions`
- **Créer un jeu** : `POST /games`
- **Terminer une session** : `PUT /sessions/:sessionId`

- Récupérer les détails d'un joueur : `GET /players/:name`
- Enregistrer un nouveau joueur : `POST /players`

Django Backend

- Les routes de l'API Django sont définies dans les vues de l'application Django. Actuellement, la liaison avec le frontend React n'est pas encore réalisée.
-

8. Composants

- **DiceGame.js :**
 - Gère le jeu de dés.
 - Affiche le nom du joueur, la valeur actuelle du dé, et les boutons pour lancer le dé ou passer le tour.
 - Enregistre les scores dans `localStorage` et met à jour la base de données via des appels API.
 - **Score.js :**
 - Affiche les scores des différentes sessions de jeu.
 - Récupère les données de `localStorage` et les affiche dans une liste.
 - **Header.js :**
 - Affiche l'en-tête avec le menu de navigation.
 - Comprend des liens vers différentes sections de l'application.
 - **Footer.js :**
 - Affiche le pied de page de l'application.
 - Contient des informations supplémentaires ou des liens pertinents.
 - **Login.js :**
 - Génération d'un Nom aléatoire :
 - Gestion du Formulaire de Connexion
 - **SessionForm.js :**
 - en cours de développement
-

9. Fonctionnalités

- **Lancer un dé et enregistrer le score :** Le joueur peut lancer un dé, et le score est enregistré pour la session en cours.
 - **Passer son tour :** Le joueur peut passer son tour, enregistrant un score de 0.
 - **Terminer une session :** Le joueur peut terminer une session, et les scores de toutes les parties de cette session sont affichés.
 - **Voir les scores de toutes les sessions terminées :** Les scores sont stockés et peuvent être consultés ultérieurement.
-

10. Fonctionnalités en Cours de Développement

- L'espace d'administration pour configurer les parties.
 - La finalisation des fonctionnalités de l'API Django pour l'intégration avec le frontend.
 - Le mode multijoueur
-

11. Tests

- Les tests pour le frontend, le backend Express et le backend Django doivent être développés et exécutés pour assurer la qualité du code.
 - Des tests unitaires et d'intégration seront ajoutés prochainement.
-

12. Contribution

Les contributions sont les bienvenues ! Pour participer, veuillez suivre ces étapes :

1. Forker le dépôt.
2. Créez une branche pour votre fonctionnalité ou correction de bug.
3. Faites des commits clairs et descriptifs.
4. Soumettez une pull request avec une description des modifications.